Optimizing City Evacuation Plans using the Genetic Algorithm Metaheuristic

Alexander Ehrenhöfer and Damian Atlaß

Freie Universität Berlin *

This report presents an implementation of a genetic algorithm designed to solve a city evacuation problem. Genetic algorithms are one of many metamathematics able to solve problems that are to complex to be with traditional methods and here we are implementing one to guide citizens form residential areas (RAs) to points of rescue (RP) in the most optimal way without overstepping the street and PR capacities. Apart from standard genetic operators (crossover, mutation) a repair mechanism tries to correct a selection of the best solution candidates to ensure feasible end results. Parent selection is performed using a loss-based roulette function, enabling exploration of potentially beneficial traits in less fit individuals. Challenges included designing an effective loss function and tuning configuration parameters. Results show that integrating genetic algorithm principles yields improved solutions, particularly evident in the small dataset where tuning was more feasible.

1 Introduction

This report, which is dedicated to the *metaheuristics* part of this course, evaluates the implementation of a genetic algorithm whose purpose is to solve a city evacuation problem. The following pages will summarize the objective, go into details of our implementation and evaluate the results based on the requirements of the task. The project's code can be found in our github repository.

2 Problem Description

2.1 The City Evacuation Scenario

The evacuation problem consists of a number of residential areas (RAs) where a specific amount of people need to be guided to specific points of rescue (PR). These

* Report for Networks and Metaheuristics at Freie Universität Berlin. Both Authors are main-students at Technische Universität Berlin.

people, even when they are in different RAs, start at the same time, if they have been grouped in the same cluster. Clusters typically span over multiple RAs and contain specific individuals of that cluster. Individuals of a cluster do not necessarily share the same target PR.

2.2 Constraints and Requirements

Two main requirements have to be fulfilled in order for a solution to be valid:

1. the street capacity cannot be exceeded
2. the PR capacities cannot be exceeded

Additionally, the evacuation plan should be as short as possible to move people to safety quickly. The street capacity is set to 20% in our implementation, but can be set to a different value in the configuration files.

2.3 Evaluation Metrics

The main criteria for the validity of results will be the compliance of the constrains: PR capacity and street capacity. Unfortunately we cannot compare the results to other heuristics, but we will further take the runtime into consideration when evaluating the solutions.

3 Genetic Algorithms

Genetic algorithms are optimization methods inspired by natural evolution. They can solve complex problems by evolving a population of possible solutions over time to find better ones. A solution's fitness is rated by a loss function and during runtime of the algorithm, those loss values will (mostly) continuously be lowered while better solutions are approached. At the same time, one of the biggest advantages of genetic algorithms are randomized procedures, namely crossover and mutation, to explore a wider range of solutions and to not get stuck in local minima.

Further explanation of genetic algorithms can be found in Ólafsson (2006), p. 7.

3.1 Our Implementation

The GA follows a standard evolutionary approach with a fixed but configurable population size. It iteratively generates new candidate solutions by applying mutation operators.

It is important to mention, that with each generation, we do not necessary generate feasible solutions. The core idea is, that with the help of our loss / fitness function, the algorithm is able to prefer feasible solutions over non-feasible solutions and moves to a state of only optimizing these feasible solutions while instantly eliminating the non-feasible ones.

We also added a repair layer that repairs a given selection of individuals (more on this later) to achieve feasibility in early stages already.

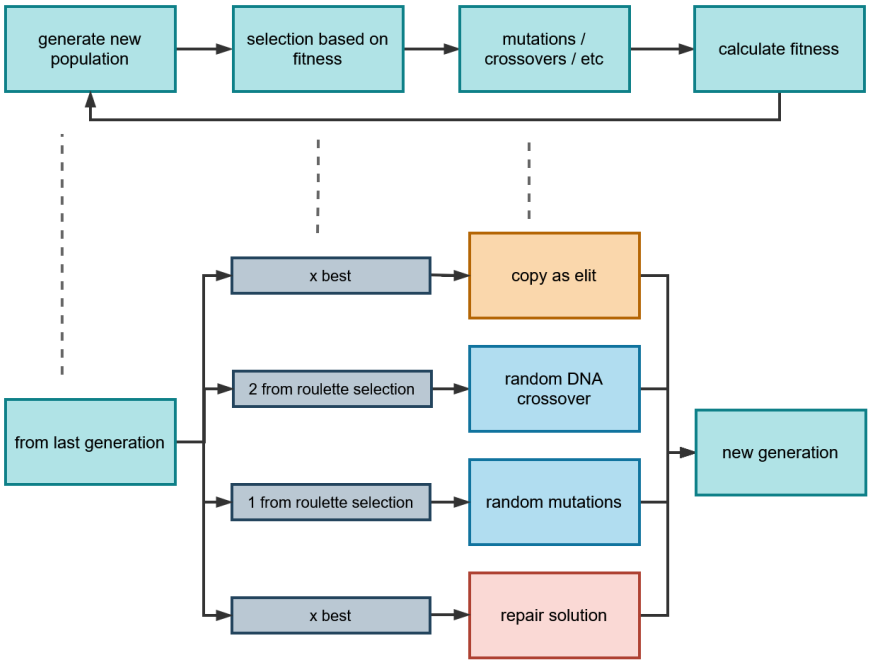


Figure 1: The basic process of our algorithm

Representation of Individuals (DNA)

Since in genetic algorithm terminology a generated solution is called an *individual* but we also deal with actual individuals (persons in the city) in our problem scenario, we will mostly use individuals to refer to individual solutions of a generation of the genetic algorithm or explicitly use the term `PossibleSolution` as we use it in the code.

Each individual (`PossibleSolution`) contains:

- A list of Route objects: Defines the evacuation route of a group of persons from a RA to a PR. This also contains the information about which cluster the group belongs to therefore holds the according starting time.
- A set of clusters: groups of RAs sharing the same start time
- The starting time for each cluster

With this data structure we are able to have all information about a possible solution in one list that can be mutated / crossed and exported in any format.

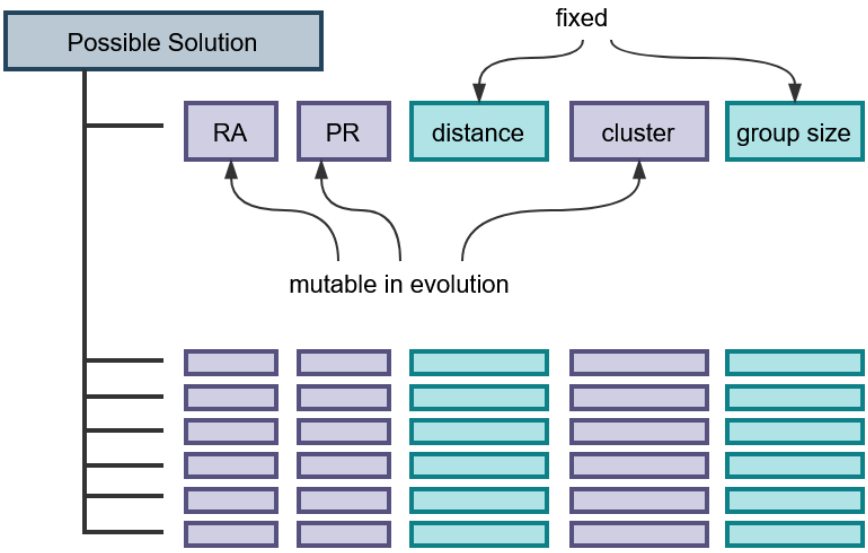


Figure 2: The DNA of our algorithm individual (We call it *Possible Solution*)

Fitness (Loss) Function

The fitness function evaluates the quality of a candidate solution using three components:

- Street capacity overflow: the sum of persons above the capacity limit on each time step
- PR (Place of Rescue) overflow after the evacuation is complete
- Total evacuation time: note that this has nothing to do with computing time but refers to the amount of *steps* that need to be taken for a given evacuation plan given each persons starting time and route distance. We set the step time to a typical person walking speed of 4 km/h.

To ensure feasibility, any overflow of street or PR capacities is penalized. Overflow values are normalized with respect to the total population and capacities.

The loss terms are defined as follows:

$$\text{Loss}_{\text{street}} = \begin{cases} 0 & \text{if there is no street overflow} \\ 40 \cdot \text{overflow} + \frac{1}{20} \cdot \text{time} & \text{otherwise} \end{cases}$$

$$\text{Loss}_{\text{PR}} = \begin{cases} 0 & \text{if there is no PR overflow} \\ 4 \cdot \text{Loss}_{\text{PR}} + \frac{1}{20} \cdot \text{time} & \text{otherwise} \end{cases}$$

The time loss is computed as:

$$\text{Loss}_{\text{time}} = \text{time} * \alpha_t$$

$$\text{TotalLoss} = \text{Loss}_{\text{street}} + \text{Loss}_{\text{PR}} + \text{time}$$

α_t is a parameter that regulates the importance of improvements in terms of time. Setting it to low can cause the algorithm to finish early without exploring the solution

space and setting it to high can lead to invalid results regarding street capacity and rescue area capacity. A higher number of datasets could help transforming this parameter into a function, by deriving its value from the input data, but since there are only three datasets, each configuration file has a fine-tuned value of its own.

Genetic Operators

Selection: The top-N possible solutions are being selected for the next generation using the roulette function (Ólafsson (2006), p. 7) based on our loss scores, the selection process for the different mutations can be seen in figure 1.

Mutation: Several mutation strategies are applied:

- Reassigning RAs to different clusters randomly or via population balancing
- Moving cluster start times backwards or forwards within a fixed window
- Reassigning individual evacuees to a different PR (based on weighted probabilities calculated by the distance to each PR)

When mutating the possible solutions we apply random mutation ranges for both the probability if a given mutation happens and how strong the mutation is applied (e.g. how far a starting time will be moved) to ensure both exploration and exploitation happen in any evolutionary epoch.

Besides these mutation, we also apply a random DNA crossover to a given amount of possible solutions as well as the typical elitilism to never loose good solutions.

Repair Mechanism

After mutation, which does automatically create feasible solutions, individuals may be invalid due to constraint violations like too many persons on the street or overflown PRs. To handle this, we added repair functions:

PR Overflow Repair: Routes going to overflown PRs are reassigned to under-utilized PRs based on precomputed weighted edges.

Street Overflow Repair: Evacuation start times of routes are delayed if too many people are simultaneously on the streets.

To ensure a broad range of possible solutions for upcoming generations, we do not apply this repair mechanism on every possible solution but just on a given selection.

3.2 Results of our Genetic Algorithm

To evaluate the performance of our algorithm, we added multiple plot methods to the export. We plot the amount of people on the street at each point of time, the final PR usage vs. capacity, a generation loss (population average as well as best possible solution), a detailed line chart for each individuals best solution loss per epoch to analyze if the losses are balanced well, and a scatter plot which shows the loss of each

individual of each generation while indicating its birth type (mutation / crossover / elite / reparation). Furthermore, we also plot a visualization of the final evacuation plan in the form of a GANTT timeline.

3.2.1 The small dataset

The *small_evacuation_data.json* dataset of the 4 provided ones provides data for roughly 2700 citizens in 113 RAs who need to be distributed to 18 different RP with a capacity of 3300. Below are the loss charts for running our algorithm on this:

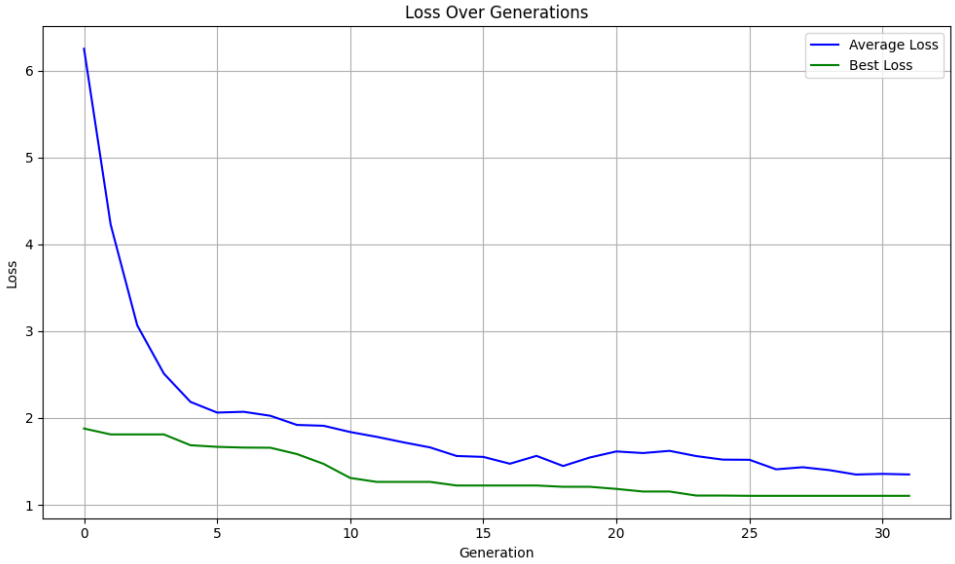


Figure 3: Average and best loss of each generation. The losses confirm that the algorithm has a decent learning curve.

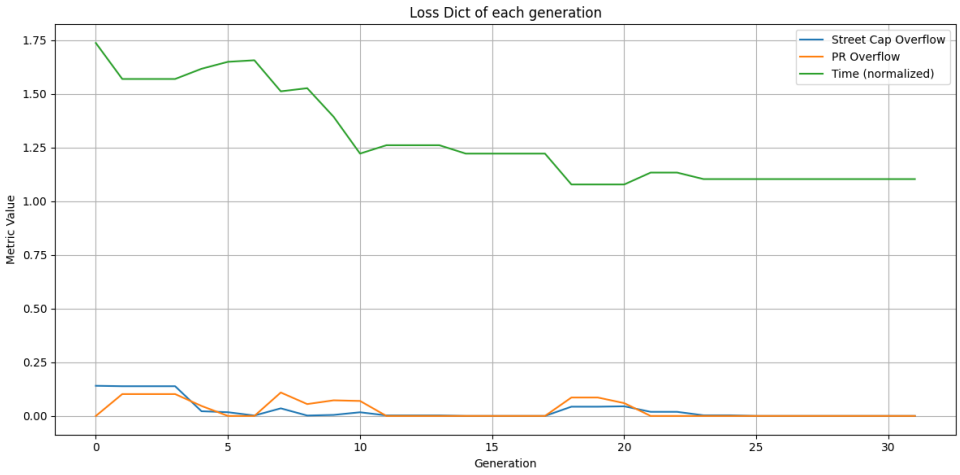


Figure 4: The individual losses of the best Possible Solution of each epoch. The graph shows that mutation and crossover can overburden the street and RP capacity, but finds a way back to a valid solution.

Figure 3 shows that the algorithm is gradually reducing loss over all both average and best loss in each iteration.

While the first valid solution was produced at iteration 6, letting the algorithm continue offers opportunity for a better result. During the following iterations, PR overflow and street capacity are exceeded as a result of the crossover or mutation phase and are carried into the following iterations in this state while reducing $Loss_{\text{time}}$. This also highlights on of the downsides of genetic algorithms: while assuming that even bad solutions carry good parts, there might be iterations where the "best" solution is invalid. This is due to that fact that punishing invalid solutions intensely might leave us in a local optima, so instead we use them with a softer penalty, allowing exploration of the search space in the hope that recombination, mutation and reparation will eventually produce valid and better-performing offspring. Figure 3 shows that the loss values are stagnating in the last third, indicating that some optimum has been found, however since the algorithm was instructed to stop when no improvement occurs after a set of iterations, an even better solution might have been computable with more time.

As it can be seen in figure 4, the generated solution is feasible regarding the overflow constraints. This can also be seen when inspecting the persons on the street visualization and the PR usage vs. capacity in figure 5:

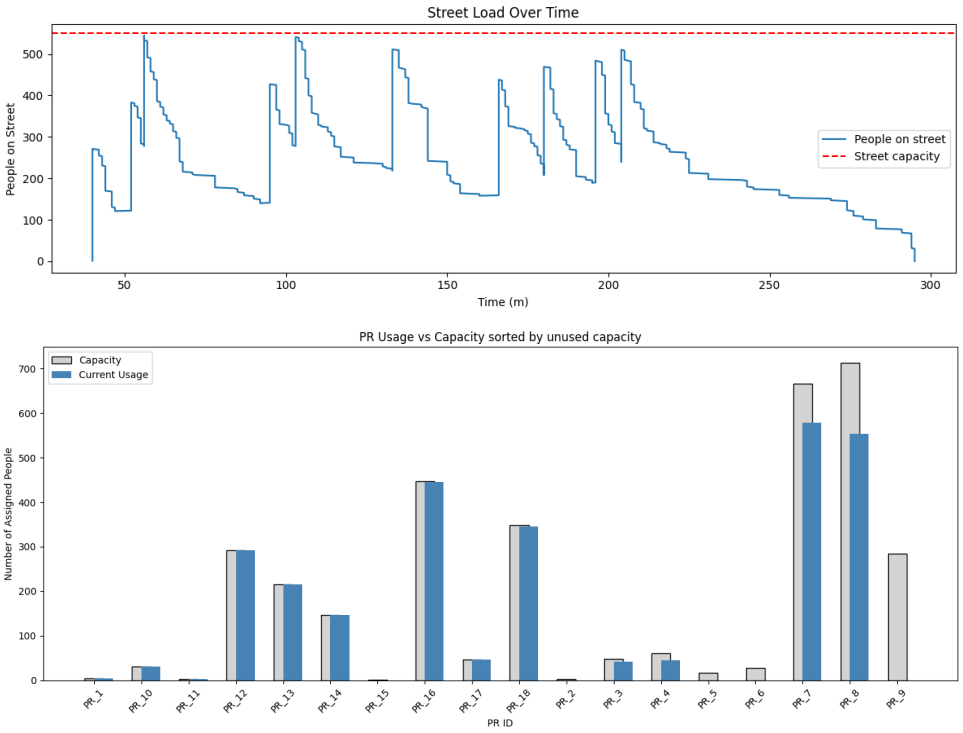


Figure 5: Visualization of the amount of people on the city streets at each timestep (top) and the final Place of Rescue usage versus it's capacity (below).

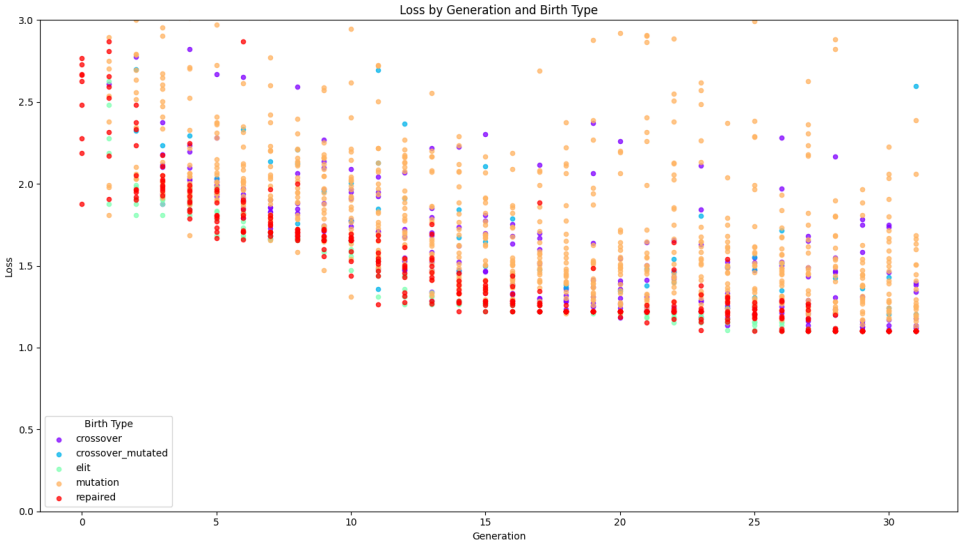


Figure 6: Visualization of loss value per individual per generation combined with type of creation.

Figure 6 displays the birth-type and loss of each individual solution in a generation. It can be seen that repaired solutions generally perform best but are occasionally bested by mutated, unaltered solutions. Although the deviation gradually decreases, solutions with higher loss are still generated to encourage exploration of the search space.

The route timeline plot can be found in the appendix in section 5.1.

3.2.2 The middle dataset

The middle-sized dataset turned to be more challenging due to its much bigger size. It dataset contains 64000 citizens in 2300 RAs who need to be guided to almost 700 RP with a capacity of 82000. The 1.6 million edges add to the difference in complexity compared to the former dataset and increase the average computation time by a multitude. While the algorithm only takes a what feels like less of a minute for the small dataset, figure 7 shows that the middle-sized dataset usually takes a lot longer.

```
Iteration time (current/avr/max/min): 16.22 / 9.37 / 16.22 / 6.79  (minutes)
```

Figure 7: Iteration runtime of the algorithm when executed on the middle-sized dataset for 10 iterations on an Intel Core i5 12500H processor adding up to about 90 minutes.

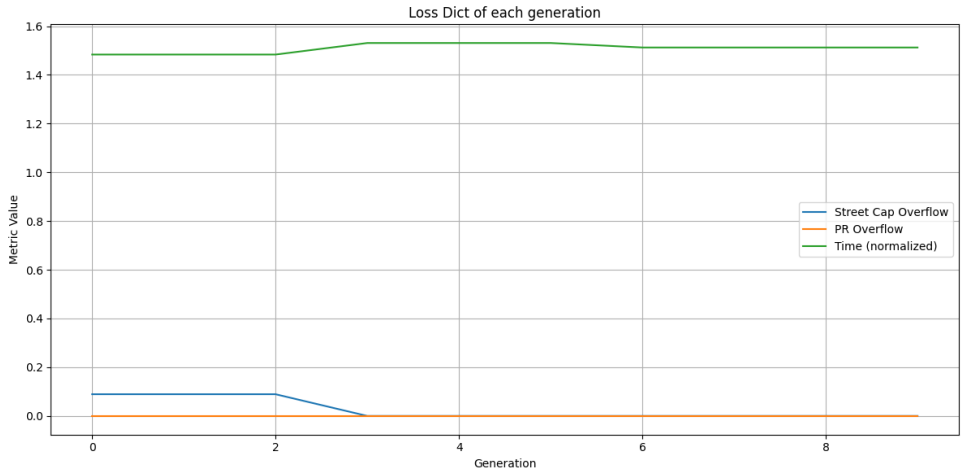


Figure 8: The individual losses of the best Possible Solution of each epoch.

Figure 8 shows that after finding a valid solution in iteration 3, the only following small change in evacuation time happens in iteration 6. The algorithms seems to struggle with finding a noticeably better solution as figure 10 shows. To achieve the runtime in figure 7 and prevent memory overflows we had to reduce the number of individuals of a generation from 50 to 30, which almost cuts the number of new possible solutions in half and might be the cause of a somewhat modest pace of improvements and why trying to increase α_{time} (and therefore punish long evacuation time harder) for this configuration is often met with invalid solutions. While the final solution is valid, the algorithm's behavior raises questions about how much untapped potential remains. Considering options to reduce runtime by identifying potential faster technics could also be a worthy time investment. Figure 9 shows how solutions generated through mutation and crossover vary a lot in their respective fitness but do not surpass the repaired solution as often as in the small-sized dataset (see figure 6).

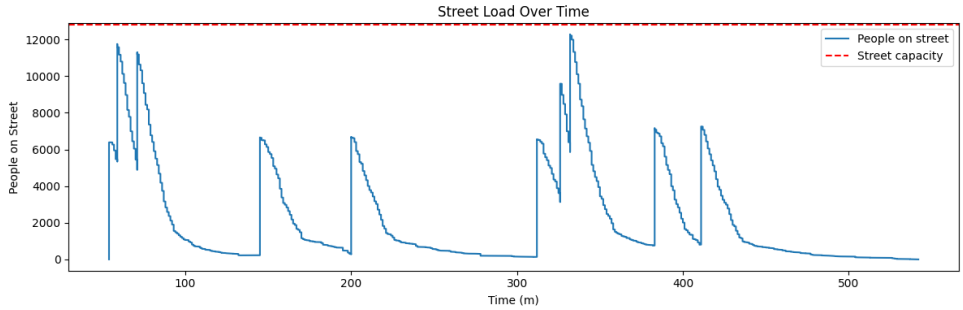


Figure 9: Visualization of the amount of people on the city streets at each timestep.

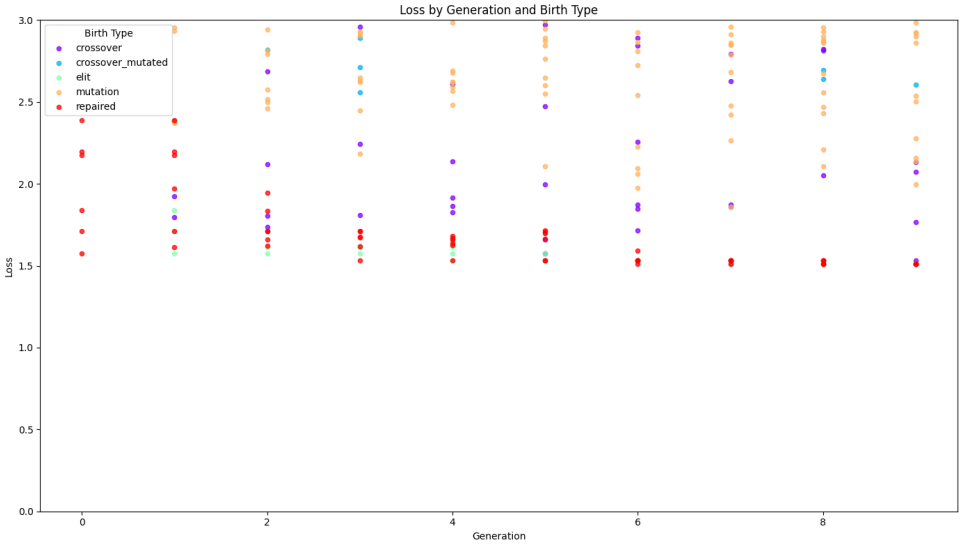


Figure 10: (Zoomed) visualization of loss value per individual per generation combined with type of creation.

3.2.3 The big dataset

Due to its massive size resulting in long computation times and tight restrictions on the number of individuals per generations due to memory constrains, experimenting on this dataset was not possible.

3.3 Limitations and Future Work

While our implemented Genetic Algorithm achieves decent results and demonstrates a clear ability to approach feasible solutions, there are several limitations and opportunities for further improvement.

Randomness in Mutation Strategy

A core weakness lies in the nature of the mutation operator. Currently, mutations are applied randomly without incorporating information about what specifically contributed to a good or bad fitness score. As a result, even though the GA can detect good candidate solutions, it is not guaranteed to optimize them further in a meaningful or targeted way. The algorithm currently lacks feedback mechanisms that help reinforce successful traits and suppress unhelpful mutations.

Although elitism is implemented to preserve the best individuals across generations, this alone does not ensure effective exploitation of promising solutions. In practice, this can lead to stagnation where improvements plateau because the algorithm cannot focus on refining known good configurations.

This limitation is especially visible in the street overflow plots (see figure **??**). For example, clusters sometimes start later than optimal. While the algorithm avoids infeasible solutions, it misses opportunities for improvement that are clearly evident

through manual inspection. These inefficiencies suggest that certain areas of the search space are under-explored, possibly due to the lack of informed local optimization.

Missing Causal Attribution

Another constraint is the absence of causal attribution within the GA. There is no mechanism that allows the algorithm to associate specific components of a solution (e.g., cluster timings, PR assignments) with their contribution to the final loss. Without this, it is difficult to apply targeted mutations that directly address the sources of overflow or inefficiency.

We think that a promising direction for future work would be to incorporate some kind of a gradient-free backpropagation or heuristics inspired by reinforcement learning. These methods could help estimate the influence of individual elements in the loss function and enable smarter, guided mutations.

Over-fitting

One risk taken when configuring an algorithm for a very limited amount of datasets is over-fitting specific parameters. Over-fitting usually refers to machine-learning-algorithms getting too well-adjusted to their training data and produce bad output when confronted with different data. Here it would apply to the loss-function or parameters in the code to be too well-adjusted the to provided datasets, which might lead to bad results with new hypothetical datasets.

Future Directions

To conclude, the following improvements are proposed for a improved future implementation:

- **Guided Mutation:** Introduce scoring heuristics that prioritize mutation of high-impact routes, clusters, or assignments based on their contribution to loss components.
- **Dynamic Mutation Rate:** Adjust mutation intensity based on generation progress or stagnation detection (At the moment the mutations have the exact same random range throughout every epoch).
- **Overflow-Aware Scheduling:** Integrate post-processing repair methods more tightly into the evolution cycle to avoid street and PR overflows more effectively.

Computation time and memory constrains

Configuring an algorithm that takes hours to run can be challenging, which is why our algorithm yields better results on the small dataset. Additionally, as mentioned before, RAM constrains do not allow us to apply the same number of individuals per generations with with middle-sized dataset as compare to the small one and prohibits us from efficiently working with the big dataset.

4 Conclusion

Our genetic algorithm successfully uses traditional mechanisms like crossover and mutation as well as a repair mechanism to create new generations of solutions for each iteration. Every new generation's parents are selected by a loss-value based roulette function to make used of the genetic algorithms main strength: occasionally selecting less fit solutions allows the algorithm to explore potentially valuable traits they may carry, thereby increasing the chance of finding better solutions. Challenges included finding a suitable loss function and configuration parameters. Results indicate that, even though the repair mechanism finds good solutions on its own, solutions closer to an optimal can be found by making use of the base principles of genetic algorithms. This is especially clear to see when executing it on the small dataset, as high computation times made adjusting configuration parameters form the middle-sized dataset difficult.

5 Appendix

5.1 Appendix: Further Result Charts of the Genetic Algorithm

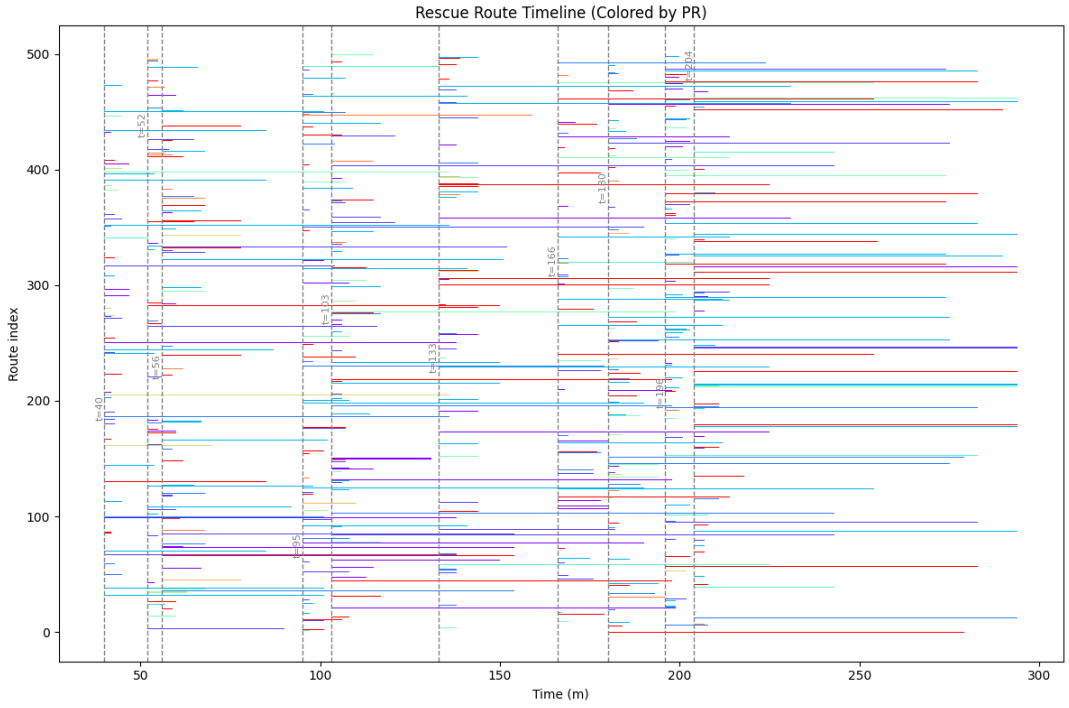


Figure 11: An export of the final generated evacuation plan for the small dataset. Each color represents a RA while the vertical lines show the starting point of a cluster of the small-sized dataset.

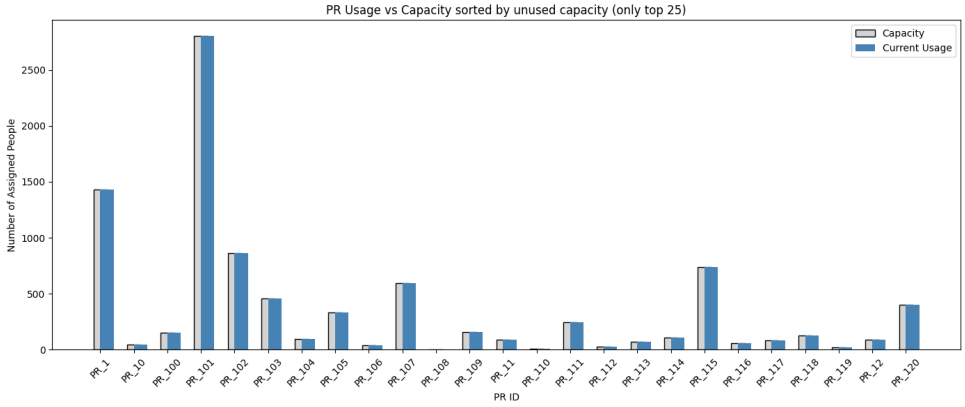


Figure 12: Final top 25 Place of Rescue usage versus its capacity when sorted by unused capacity for the middle-sized dataset. If at least one overflow existed, it would be shown on the very left bars.

List of Figures

1 The basic process of our algorithm . . . 3
2 The DNA of our algorithm individual (We call it *Possible Solution*) . . . 4
3 Average and best loss of each generation. The losses confirm that the algorithm has a decent learning curve. . . . 6
4 The individual losses of the best Possible Solution of each epoch. The graph shows that mutation and crossover can overburden the street and RP capacity, but finds a way back to a valid solution. . . . 6
5 Visualization of the amount of people on the city streets at each timestep (top) and the final Place of Rescue usage versus it's capacity (below). . 7
6 Visualization of loss value per individual per generation combined with type of creation. . . . 8
7 Iteration runtime of the algorithm when executed on the middle-sized dataset for 10 iterations on an Intel Core i5 12500H processor adding up to about 90 minutes. . . . 8
8 The individual losses of the best Possible Solution of each epoch. . . . 9
9 Visualization of the amount of people on the city streets at each timestep. 9
10 (Zoomed) visualization of loss value per individual per generation combined with type of creation. . . . 10
11 An export of the final generated evacuation plan for the small dataset. Each color represents a RA while the vertical lines show the starting point of a cluster of the small-sized dataset. . . . 12
12 Final top 25 Place of Rescue usage versus its capacity when sorted by unused capacity for the middle-sized dataset. If at least one overflow existed, it would be shown on the very left bars. . . . 13

References

Ólafsson, S. (2006). Chapter 21 metaheuristics. In S. G. Henderson & B. L. Nelson (Eds.), *Simulation* (pp. 633–654, Vol. 13). Elsevier. https://doi.org/https://doi.org/10.1016/S0927-0507(06)13021-2